Call through a base pointer and C++ picks the function by the pointer's type. Unless it is virtual.

```
Animal* a = new Dog;
a->speak();     // which speak() runs?
```

The answer surprises most beginners.

swipe →

Without `virtual`

```
struct Animal {
    void speak() { puts("..."); }
};
struct Dog : Animal {
    void speak() { puts("Woof"); }
};

Dog d;
Animal* a = &d;
a->speak();
```

Prints the Animal version, not Woof.

The compiler only knows `a` is an `Animal*`, so it calls `Animal::speak`.

With `virtual`

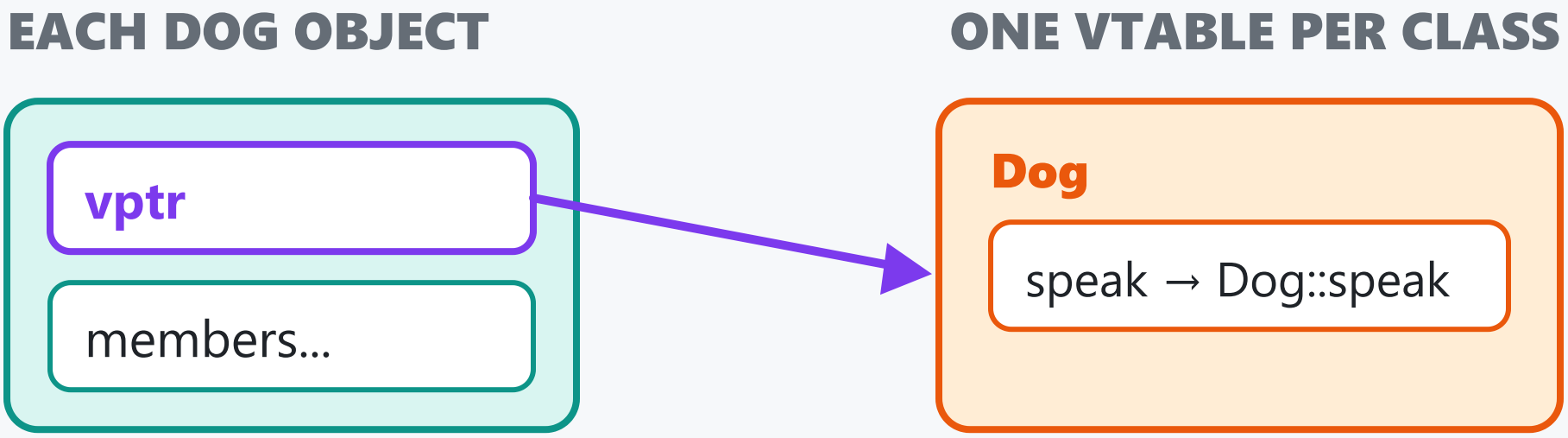
```
struct Animal {
    virtual void speak() { puts("..."); }
};
struct Dog : Animal {
    void speak() override { puts("Woof"); }
};

Animal* a = &d;
a->speak();
```

Prints `Woof`.

Now the function is chosen by what the object **really is**, at run time.

How it works



The call follows the arrow at run time.

A class with virtual functions gets a table of them. Every object carries a hidden pointer to its class's table.

What you get

```
std::vector<std::unique_ptr<Animal>> zoo;
zoo.push_back(std::make_unique<Dog>());
zoo.push_back(std::make_unique<Cat>());

for (auto& x : zoo)
    x->speak();
```

OUTPUT

```
Woof
```

```
Meow
```

One loop. Each object does its own thing. Add a `Bird` later and the loop does not change.

The small print

- mark overrides with `override`: the compiler catches typos
- give the base a `virtual` destructor
- cost: one hidden pointer per object, one extra jump per call
- it only works through a pointer or reference

Do you mark every override with `override`? It is free.